[ACCOUNT LAYER WEEK / DAY 03]

Real-time Sui.

A payment happens.
Your app should know.

GRAPHQL SUBSCRIPTIONS / THE NEW DATA STACK

SUIHUB LAGOS / ACCOUNT LAYER WEEK

An account becomes an experience.

DAY 01

Identity

Who can access the account?

DAY 02

Permission

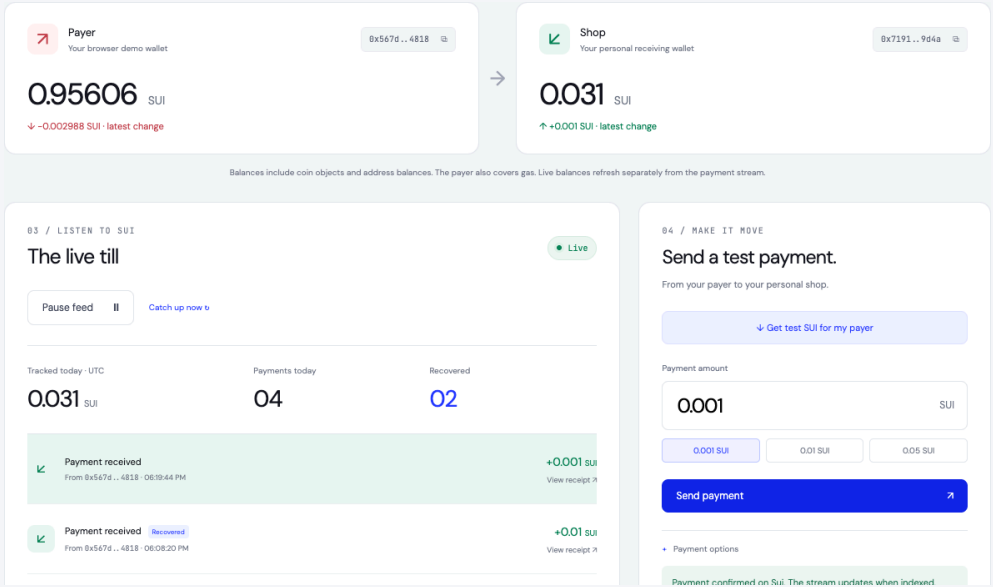
What may a spender do with it?

DAY 03

Visibility

How does your app see what happened?

Meet your live till.



PERSONAL

Your payer. Your shop.

LIVE

A receipt from the stream.

RECOVERABLE

A saved place to resume.

Live app: day3-till.vercel.app / Real testnet transactions. Personal student wallets.

Build it. Break the connection. Recover.

01

Choose

gRPC, GraphQL, archive or indexer

02

Listen

Filtered subscriptions over POST

03

Recover

Cursors, ordering and replay

04

Keep

SQLite totals and old receipts

The final test: refresh the page without counting a payment twice.

Four layers. Four different jobs.

01 / gRPC

Read and act now.

Balances, objects, execution and checkpoints.

02 / GRAPHQL

Ask for a screen.

Filtered history, selected fields and subscriptions.

03 / ARCHIVE

Keep the past.

History beyond a full node’s retention.

04 / YOUR INDEXER

Own the data model.

Durable aggregates, joins and reprocessing.

Choose for the job you are doing.

WALLET

gRPC + history

Execute now. Page activity with GraphQL.

SHOP

Subscription + writer

Receive payments. Keep a durable ledger.

GAME

Actions + event feed

Execute with gRPC. Stream game events.

ANALYTICS

Checkpoint indexer

Backfill, transform and own the database.

The migration is already real.

JULY 2026

JSON-RPC disabled

Foundation mainnet rollout began the week of 27 July.

21 SEPTEMBER 2026

Subscriptions live

GraphQL subscriptions announced for mainnet and testnet.

15 OCTOBER / TESTED

Check the body.

Public testnet JSON-RPC returns an error inside HTTP 200.

Push updates.
Keep a place to resume.

A live feed is only useful if it can survive a disconnect.

One document. Any shop.

```
subscription Till($shop: SuiAddress!, $after: String, $afterCheckpoint: UInt53) {
 transactions(
   filter: { affectedAddress: $shop, afterCheckpoint: $afterCheckpoint }, after: $after
 ) {
   cursor
   node {
     digest sender { address }
     effects {
       status timestamp checkpoint { sequenceNumber }
       balanceChanges(first: 50) { nodes { owner { address } coinType { repr } amount } }
     }
   }
 }
}
```

Select a small payload. Every matching transaction carries a cursor.

Filter the activity you care about.

TRANSACTIONS

Who or what?

affectedAddress · affectedObject
function · kind · sentAddress

EVENTS

Which event?

sender · module · type
plus checkpoint boundaries

CHECKPOINTS

Progress through Sui.

Follow checkpoint progress.
Resume with a saved position.

The transport is an HTTP POST.

```
const res = await fetch(url, {
   method: 'POST',
   headers: { 'content-type': 'application/json', accept: 'text/event-stream' },
   body: JSON.stringify({ query, variables }), signal,
 });
 if (!res.ok || !res.body) throw new Error(`Subscription HTTP ${res.status}`);
 if (!res.headers.get('content-type')?.includes('text/event-stream')) {
   throw new Error('Expected an event stream');
 }
 const reader = res.body.pipeThrough(new TextDecoderStream()).getReader();
```

POST /graphql/subscriptions / Content-Type: text/event-stream / Browser and Node

An address match is not yet a sale.

```
if (node.effects.status !== 'SUCCESS') return [];
const me = normalize(shop);
const net = new Map<string, bigint>();
for (const c of node.effects.balanceChanges.nodes) {
 if (c.owner && normalize(c.owner.address) === me) {
   const [address, ...rest] = c.coinType.repr.split('::');
   const type = [normalize(address), ...rest].join('::');
   net.set(type, (net.get(type) ?? 0n) + BigInt(c.amount));
 }
}
```

Success → net changes for this shop → positive receipts → format for display

Every student gets their own workspace.

01

Choose

Demo payer or a connected testnet wallet

02

Fund

Get test SUI for the payer

03

Open

Start listening before sending

04

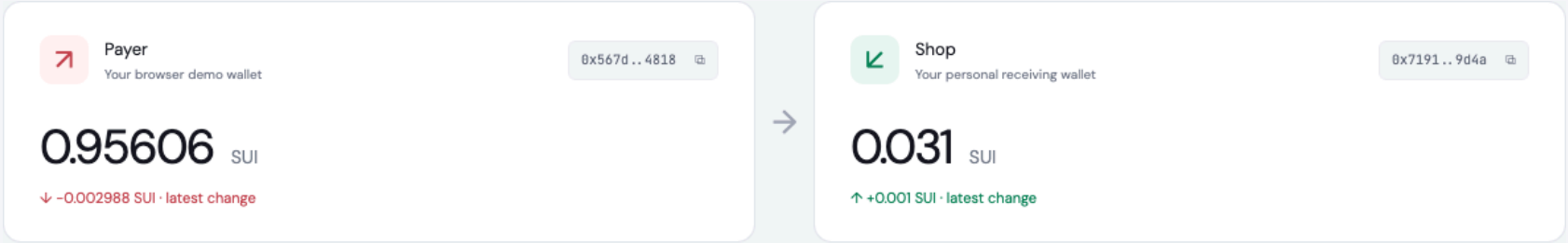
Pay

Send 0.01 SUI to your own shop

Watch the money move.

PAYER / MONEY OUT

SHOP / MONEY IN



Payment + gas

Payment received

Payer ↓ payment + gas / Shop ↑ received value / Balances refresh independently of the feed

First, prove the normal path.

01

Open till

Wait for the Live badge

02

Send

Pay 0.01 test SUI

03

Observe

Balance change + incoming receipt

04

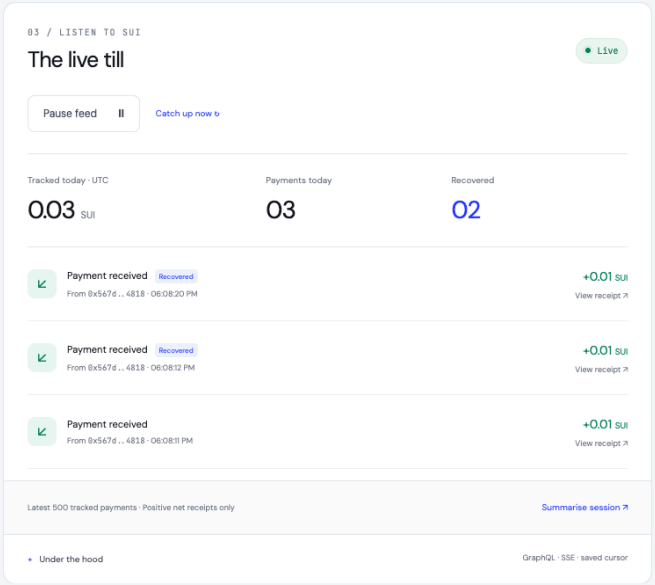
Inspect

Open the transaction receipt

Pause the feed.
Send another payment.

The chain keeps going. Only your listener stops.

It catches up from where you stopped.



01 / PAUSE

The listener stops.

02 / PAY

The chain keeps going.

03 / RESUME

Missed receipts return.

Saved position → historical catch-up → live delivery on the same connection

No cursor yet? Keep a checkpoint.

01

First open

Capture an opening checkpoint

02

First delivery

Save the cursor with its payment

03

Reconnect

Use after, or the opening checkpoint

04

Continue

Replay first. Then follow live.

Save a receipt and its position together.

```
const seen = new Set(previous.payments.map(p ⇒
   `${p.digest}:${p.coinType}`));
 const added = paymentsFrom(delivery, shop)
   .filter(p ⇒ !seen.has(`${p.digest}:${p.coinType}`))
   .map(p ⇒ ({ ...p,
     recovered: Date.parse(p.timestamp) < recoverBefore }));

// Save the cursor and payments in one write.
localStorage.setItem(snapshotKey(shop), JSON.stringify(next));
```

A replay is allowed. Double counting is not.

Delivery has edges. Design for them.

ORDER

Use chain order.

Checkpoint, then transaction position. Events add event position.

RETRY

Back off and resume.

Cancel old timers and streams when the shop changes.

FILTERING

Check both paths.

A live filter and historical replay can behave differently.

CAPACITY

Expect throttling.

Per-subscriber limits apply. Do not assume infinite throughput.

A live feed can become a small indexer.

01

Subscribe

The same document and parser

02

Begin

Start a database transaction

03

Write

Insert receipts + save watermark

04

Commit

Advance both, or neither

The cursor belongs in the same commit.

```
db.exec('BEGIN');
try {
 for (const p of payments) {
   insert.run(p.digest, p.coinType, p.amount,
     p.sender, p.checkpoint, p.timestamp, p.cursor);
 }
 saveCursor.run(shop, d.transactions.cursor,
   new Date().toISOString());
 db.exec('COMMIT');
} catch (e) {
 db.exec('ROLLBACK');
 throw e;
}
```

PRIMARY KEY (digest, coin_type) / Amounts as TEXT / Watermark per shop

Run the writer. Restart it. Compare totals.

```
pnpm install
cp .env.example .env
# Set SHOP_ADDRESS to the address shown in your Till.
# Set FROM_CHECKPOINT to its opening checkpoint.
pnpm indexer

# Stop and restart the writer. Then, in another terminal:
pnpm totals
pnpm receipt
```

SQLite runs locally. The Vercel app does not host this long-running writer.

Know when the small writer is too small.

LIGHTWEIGHT WRITER

One focused feed.

A shop address, modest volume and a simple derived table.

CUSTOM INDEXER

A chain-scale pipeline.

Checkpoint backfills, parallel work, schema ownership and reprocessing.

A full node can forget. History remains.



Payment record

SUCCESS

Digest	ArCwXsZUTywBuM4GbNX4CVMERmBBgeYY2mgmJJjdHa3a ↗
Recorded	Tue, 07 Oct 2025 22:04:44 GMT
Checkpoint	250,000,000

Balance changes

0xcdf2..4eb8	-0.006844038 SUI
0xcdf2..4eb8	+0.00000017 nBTC

Source: Sui GraphQL. A sender's net change can include gas.

DATE

7 October 2025

CHECKPOINT

250,000,000

RESULT

GraphQL found the receipt.

Verified 15 October 2026 / Transaction dated 7 October 2025 / Checkpoint 250,000,000

Design inside the service limits.

PAGINATION

20 default / 50 max

Observed for Query.transactions. Page through longer history.

QUERY BUDGET

40 seconds

Observed query timeout. Use small field selections.

PRODUCTION

Choose a provider.

Check quotas, retention, auth, uptime and subscription support.

Migrate the intention, not just the method.

```
pnpm migrate:legacy   # inspect the JSON-RPC error body
pnpm migrate:grpc     # current balance through gRPC
pnpm migrate:graphql  # structured indexed query

// gRPC SDK shape used by the app:
const { balance } = await client.getBalance({
 owner: address, coinType: '0x2::sui::SUI'
});
// balance.balance includes coin and address balances.
```

Execute and read current state with gRPC. Query indexed history with GraphQL.

The lifecycle is part of the demo.

REFRESH

Resume explicitly.

Open till restores the saved per-shop position.

SWITCH ACCOUNT

Switch the shop too.

A connected payer gets its own personal receiving address.

RESET SESSION

Clear tracking only.

Existing onchain funds and wallet keys stay in place.

UPGRADE

Treat cursors carefully.

Version local state. Rebuild if query semantics change.

Your turn: prove all four.

01

Isolation

Two browsers show different shops

02

Payment

A receipt matches a real digest

03

Recovery

Pause, pay, resume and reload

04

History

Find the October 2025 receipt

Explain why the payer can lose slightly more SUI than the shop receives.

Make it live.
Make it recover.

day3-till.vercel.app

github.com/Nuel-osas/day3-till

YOUR NEXT BUILD / A LIVE FEED WITH A MEMORY